

Portfolio

김승환 · 기술 포트폴리오

이력서와 함께 제출하는 구현 사례 문서

메디게이트는 [회사 공식 공개 자료](#) 기준(2026.06) 의사 회원 130,963명이 가입한 서비스입니다. 커뮤니티 서비스 리뉴얼을 시작한 2024.10부터 FE 파트리더 역할을 맡고 있습니다.

아래에는 이력서의 경력 목록 대신 인증, 리포트, 모바일 상태와 디자인 토큰 구현을 정리했습니다.

사례 1. 여러 요청이 동시에 만료될 때 refresh 한 번만 보내기

대상: 의사 대상 포털 리뉴얼

역할: FE 파트리딩 및 프론트엔드 개발

기술: Next.js 14, React 18, TypeScript, Ky

JSP와 Next.js 화면이 함께 운영되는 동안 기존 인증 API를 그대로 사용해야 했습니다. 한 화면에서 보낸 여러 API가 동시에 401을 반환하면 요청마다 refresh를 보내는 문제가 생길 수 있었습니다.

구현

첫 401 요청이 만든 갱신 작업을 HTTP 모듈에 보관했습니다. 뒤이어 401을 받은 요청은 새 갱신을 시작하지 않고 같은 작업의 완료를 기다립니다. 성공한 경우에만 원래 요청을 다시 보내고, 실패하면 기존 401 처리로 넘깁니다.

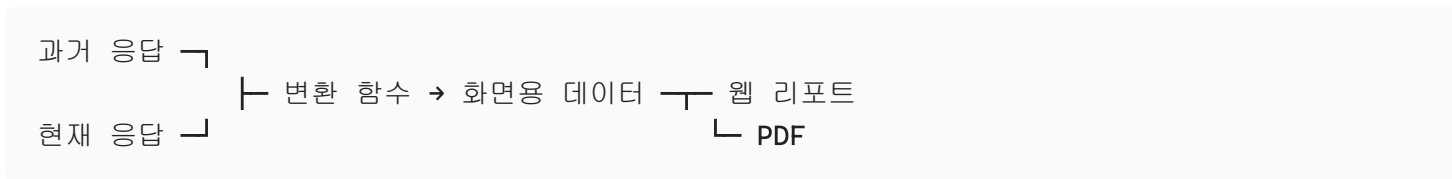
대상: 개원입지·상권 분석 서비스

기술: Next.js 13–14, TypeScript, TanStack Query, Zustand, Naver Maps API, Puppeteer, pdf-lib

사용자가 주소, 역, 반경과 진료과를 바꾸면 지도와 분석 리포트가 함께 달라집니다. 과거에 구매한 리포트와 현재 리포트의 응답 필드가 달랐고, 같은 내용을 웹 화면과 PDF에서 보여줘야 했습니다.

데이터 경계

첫 화면에 필요한 값과 지도 조작 뒤 필요한 값을 나눠 API 호출 시점을 정했습니다. 과거 응답과 현재 응답은 렌더링 전에 화면용 데이터로 변환했습니다. 화면과 PDF는 변환이 끝난 같은 데이터를 사용합니다.



PDF 생성은 브라우저 준비, 페이지 이동, 렌더링과 파일 병합 단계로 나눴습니다. 브라우저는 다시 사용하되 열린 페이지가 쌓이면 재시작합니다. 각 단계에 로그를 남겨 완료되지 않은 위치를 찾을 수 있게 했습니다.

운영 영향

- 과거와 현재 리포트에 화면별 조건문을 흘뿌리지 않고 변환 함수에서 응답 차이를 처리했습니다.
- 웹과 PDF가 같은 화면용 데이터를 사용해 두 출력의 조건을 한곳에서 확인합니다.
- PDF가 멈추면 준비, 이동, 렌더링과 병합 중 마지막으로 진행된 단계를 로그에서 구분합니다.

감수한 비용

응답 변환 계층과 브라우저 관리 코드가 추가됐습니다. 대신 각 화면에서 과거 버전을 판별하거나 PDF 요청마다 브라우저를 새로 띄우는 방식을 피했습니다.

사례 3. 모바일 화면 이동 뒤 검색 조건과 보던 위치 복원하기

대상: 메디게이트 모바일 서비스

기술: Next.js 13, React 18, TypeScript, Redux Toolkit, TanStack Query, React Hook Form, Flutter WebView

목록에서 상세, 영상이나 작성 화면으로 이동했다 돌아오면 검색 조건과 보던 위치가 사라졌습니다. 서버에서 다시 받을 값과 화면 이동 뒤 남겨야 할 값을 같은 저장소에 넣으면 복원 시점도 엉겼습니다.

상태를 나눈 기준

데이터	저장 위치	기준
API 응답과 목록	TanStack Query	서버에서 다시 받을 수 있습니다.
검색·정렬 조건	Redux Toolkit	다른 화면 뒤에도 남아야 합니다.
상세 진입 위치	탐색 이력과 URL	사용자의 이동 경로에 붙습니다.
작성 중인 입력	React Hook Form	제출 전까지만 필요합니다.

access token만 갱신 가능한 경우 한 번 다시 발급했습니다. refresh token이나 사용자 정보가 만료된 경우에는 요청을 되풀이하지 않고 로그인 화면으로 이동했습니다. 로그인 뒤 원래 모바일 주소로 돌아오도록 절대 URL을 사용했습니다.

웹에서 배너를 닫으면 그 값을 Flutter 앱에 전달했습니다. 앱은 받은 값으로 바텀시트와 SafeArea를 같이 닫습니다.

운영 영향

- 상세·영상·작성 화면을 다녀온 뒤 검색 조건과 보던 위치를 복원합니다.
- 갱신할 수 없는 인증 상태에서는 같은 요청을 반복하지 않고 로그인 흐름으로 전환합니다.
- 웹 배너와 앱 영역이 같은 닫힘 상태를 사용합니다.

감수한 비용

상태 도구를 하나로 통일하지 않았습니

사례 4. 기존 부채를 막지 않고 새 코드부터 디자인 토큰 검사하기

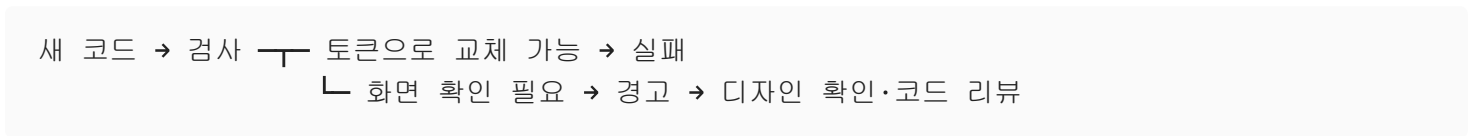
대상: 의사 대상 포털 리뉴얼

기술: TypeScript, TailwindCSS, Figma 디자인 토큰

Figma의 색상, 간격과 글자 크기 대신 비슷한 임의 값이 코드에 계속 추가됐습니다. 기존 파일 전체를 바로 실패 처리하면 새 기능 개발까지 막히기 때문에, 새로 바뀐 코드부터 검사하는 범위를 택했습니다.

검사 범위와 판정

Git diff에 추가된 줄과 새 TS·TSX 파일에서 색상 값, Tailwind 임의 값과 기본 타이포그래피 사용을 찾습니다. 토큰으로 바로 교체할 수 있으면 실패로 처리하고, 제품 화면을 봐야 판단할 수 있으면 경고로 남깁니다.



FE 팀원 3명은 같은 검사 결과를 기준으로 리뷰합니다. 디자인 담당자에게는 경고가 나온 값만 확인해 토큰 예외인지 결정합니다.

운영 영향

- 새 코드에 들어온 임의 색상과 타이포그래피를 리뷰 전에 찾습니다.
- 자동 판정할 값과 화면을 보고 판단할 값을 나눠 디자인 확인 범위를 줄입니다.
- 기존 파일 전체를 막지 않고 신규 변경부터 같은 기준을 적용합니다.

감수한 비용

기존 코드의 임의 값을 한 번에 없애지는 못합니다. 새 변경부터 더 쌓이지 않게 막고, 기존 값은 해당 화면을 수정할 때 함께 정리하는 방식을 택했습니다.

화면 문제를 서버 경계까지 확인하는 순서

Java·Spring·MyBatis로 화면, API와 SQL을 함께 개발한 경험이 있습니다. 현재 프론트엔드 문제도 아래 순서로 범위를 줄인 뒤 백엔드 개발자와 수정 지점을 나눕니다.

1. 브라우저에서 요청값과 재현 경로를 확인합니다.
2. API 상태 코드, 응답 필드와 권한을 확인합니다.
3. 여러 변경이 묶이면 Service 트랜잭션을 확인합니다.
4. 목록과 전체 건수가 다르면 두 SQL의 검색 조건을 비교합니다.

이 방식으로 프론트엔드에서 고칠 일, API 계약을 바꿀 일과 서버 내부를 확인할 일을 먼저 나눕니다.